

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Task No.: 3

Student Name: EMANDI HARI CHARAN

Roll No.: A24126552080

Date:

OBSERVATION TASK 3: DYNAMIC TO-DO LIST APPLICATION USING JAVASCRIPT DOM

Task Description & Requirements

Task Requirement:

Develop an interactive To-Do List web application using HTML5, CSS3, and JavaScript DOM manipulation. The application allows users to enter tasks into an input field, dynamically create new list items (`document.createElement`), mark items as completed (toggling CSS strikethrough classes), remove items from the DOM (`li.remove()`), and automatically toggle the visibility of an empty list message.

Source Code

`index.html`

```
<!DOCTYPE html>
<html>

<head>
  <title>My To-Do List</title>

  <style>
    body {
      font-family: Arial;
      background-color: #f2f4f8;
      text-align: center;
      margin-top: 50px;
    }

    h1 {
      color: #333333;
    }
  </style>
</head>

<body>
  <div>
    <input type="text" value="Enter your task here" />
    <button value="Add" />
  </div>
  <ul>
    <li><input type="checkbox" /> <span>Task 1</span>
    <li><input checked="" type="checkbox" /> <span>Task 2</span>
    <li><input type="checkbox" /> <span>Task 3</span>
  </ul>
  <div>
    <input type="button" value="Remove" />
    <input type="button" value="Toggle" />
  </div>
  <div>
    <div>Empty list</div>
    <div>No tasks found</div>
  </div>
</body>
</html>
```

```
.input-section {
  margin: 20px;
}

input {
  padding: 10px;
  width: 250px;
  border: 1px solid #cccccc;
  border-radius: 5px;
}

#addTask {
  padding: 10px 15px;
  background-color: #4CAF50;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

#addTask:hover {
  background-color: #45a049;
}

#emptyMessage {
  color: #777777;
}

ul {
  list-style-type: none;
  padding: 0;
}

li {
  background-color: white;
  margin: 10px auto;
  padding: 10px;
  width: 400px;
  border-radius: 5px;
  box-shadow: 0 2px 5px #cccccc;
}

.complete {
  margin-left: 15px;
  padding: 5px 10px;
  background-color: #2196F3;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.delete {
  margin-left: 5px;
  padding: 5px 10px;
  background-color: #f44336;
```

```

        color: white;
        border: none;
        border-radius: 4px;
        cursor: pointer;
    }

    .complete:hover {
        background-color: #1976D2;
    }

    .delete:hover {
        background-color: #d32f2f;
    }

    /* Completed task */
    .completed {
        text-decoration: line-through;
        color: #888888;
    }
}
</style>
</head>

<body>

    <h1>My To-Do List</h1>

    <div class="input-section">
        <input type="text" id="taskInput" placeholder="Enter a task">
        <button id="addTask">Add Task</button>
    </div>

    <p id="emptyMessage">No tasks available.</p>

    <ul id="taskList"></ul>

    <script src="Script.js"></script>

</body>

</html>

```

Script.js

```

let taskInput = document.getElementById("taskInput");
let addTask = document.getElementById("addTask");
let taskList = document.getElementById("taskList");
let emptyMessage = document.getElementById("emptyMessage");

addTask.onclick = function () {

    let task = taskInput.value;

    if (task == "") {
        alert("Please enter a task");
    }
}

```

```
        return;
    }

    // Create task
    let li = document.createElement("li");

    li.innerHTML = task;

    // Create Complete button
    let completeButton = document.createElement("button");

    completeButton.innerHTML = "Complete";
    completeButton.className = "complete";

    // Complete task
    completeButton.onclick = function () {
        li.classList.toggle("completed");
    };

    // Create Delete button
    let deleteButton = document.createElement("button");

    deleteButton.innerHTML = "Delete";
    deleteButton.className = "delete";

    // Delete task
    deleteButton.onclick = function () {
        li.remove();

        if (taskList.children.length == 0) {
            emptyMessage.style.display = "block";
        }
    };

    li.appendChild(completeButton);
    li.appendChild(deleteButton);

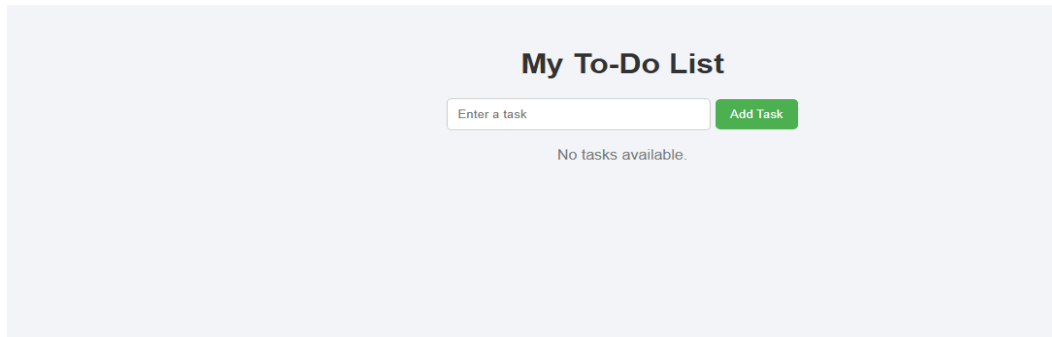
    taskList.appendChild(li);

    taskInput.value = "";

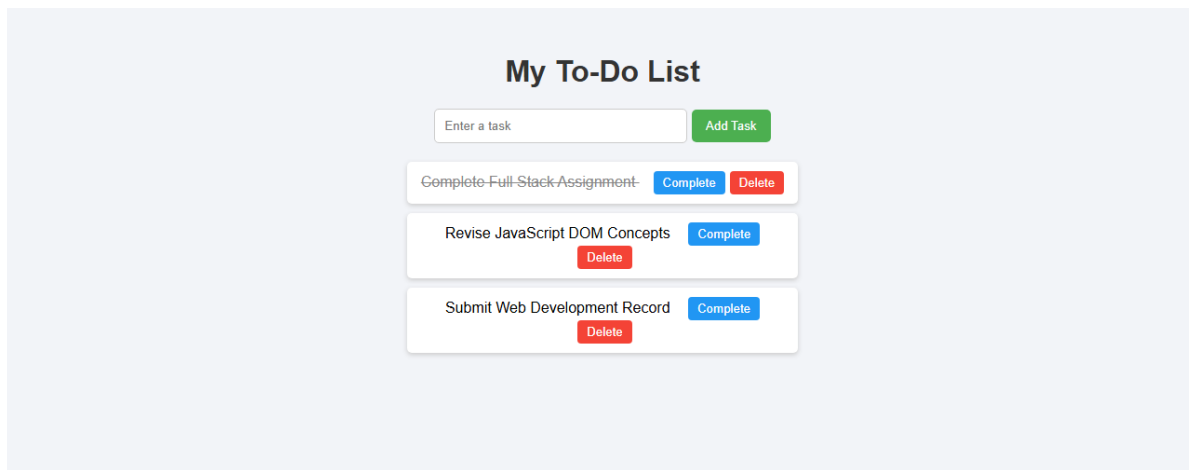
    emptyMessage.style.display = "none";
};
```

Actual Output Screenshots

Output 1: Initial Empty Task List View ('No tasks available')



Output 2: Interactive To-Do List Execution View (Adding, Completing & Deleting Tasks)



Result / Conclusion

Result: Observation Task 3 was executed and verified successfully. The dynamic To-Do List application accurately handles task entry validation, dynamic DOM element creation (and <button>), class toggling (.completed strikethrough), element removal, and empty state message toggling.